

Introduction

Single-cell RNA-seq cells differ in total UMI count by orders of magnitude — a typical 10x Chromium experiment ranges from a few hundred to tens of thousands of UMIs per cell. This library-size variation swamps biological signal if not corrected. Normalisation removes the library-size effect, and variance stabilisation goes further by flattening the mean-variance relationship so that downstream PCA and clustering are driven by biology rather than count magnitude.

Prerequisites

A working understanding of count data, library-size effects, and the negative-binomial distribution as the standard model for count overdispersion.

Theory

Three approaches dominate:

LogNormalize (Seurat default): divide each cell's count by its total, multiply by 10^4 , take $\log_{10}$. Simple, fast, and well understood; does not stabilise variance.

SCTransform: regularised negative-binomial regression with sequencing depth as the explanatory variable, producing Pearson residuals as the normalised output. Variance is stabilised by construction, and the regularised parameter pooling allows rare cell types to be recovered better than LogNormalize.

Pool-based size factors (`scran::computeSumFactors`): pool cells for robust size-factor estimation in low-depth settings, addressing the dropout problem that simple library-size normalisation does not.

Assumptions

Library-size effects are multiplicative on count expectations; counts follow a (possibly cell-specific) negative-binomial distribution with gene-specific dispersion.

R Implementation

```
library(Seurat)

set.seed(2026)
counts <- matrix(rpois(300 * 200, lambda = 2), 200, 300)
rownames(counts) <- paste0("g", 1:200)
colnames(counts) <- paste0("c", 1:300)

so <- CreateSeuratObject(counts)

# Standard LogNormalize
so <- NormalizeData(so, verbose = FALSE)
so <- FindVariableFeatures(so, nfeatures = 100, verbose = FALSE)
so <- ScaleData(so, verbose = FALSE)

# Alternative: SCTransform
so_sct <- SCTransform(so, verbose = FALSE)

head(GetAssayData(so, assay = "RNA", slot = "data")[1:5, 1:4])
head(GetAssayData(so_sct, assay = "SCT", slot = "data")[1:5, 1:4])
```

Output & Results

Two normalised matrices: LogNormalize (log-counts-per-10k stored in `data` slot of the RNA assay) and SCT (Pearson residuals stored in `data` slot of the SCT assay). Both feed the same downstream pipeline (PCA, clustering, UMAP) but produce different clustering granularity in practice.

Interpretation

A reporting sentence: “After SCTransform normalisation (`vst.flavor = 'v2'`, `vars.to.regress = 'percent.mt'`), PCA separated the major immune cell types in the first 5 components; LogNormalize required 10 components to achieve comparable separation. The SCT-based clustering identified two additional rare populations (mast cells, plasmacytoid dendritic cells) that LogNormalize merged into larger clusters.” Always state the normalisation method and any regressed-out covariates.

Practical Tips

- Use `SCTransform(vst.flavor = "v2")`, the current recommended flavour; it improves on the original SCTransform parameter regularisation.
- For rare cell types, SCTransform consistently outperforms LogNormalize; for coarse-grained analyses either method works.
- Store raw counts in the RNA assay and SCT residuals in the SCT assay; avoid re-running LogNormalize after SCTransform on the same object.
- Do not run `NormalizeData()` after `SCTransform()`; the latter already produces normalised values, and stacking is incorrect.
- For multi-sample integration, normalise each sample independently before merging (`SCTransform` per sample, then `IntegrateData` or `harmony`); pooling raw counts then normalising loses sample-specific structure.
- Regress out unwanted technical covariates (mitochondrial percentage, cell-cycle scores) via `vars.to.regress` in `SCTransform`; this often improves downstream clustering.

R Packages Used

`Seurat::NormalizeData()` and `SCTransform()` for the canonical scRNA-seq normalisation methods; `scran::computeSumFactors()` for pool-based size factors in the Bioconductor ecosystem; `sctransform` (the underlying engine) for low-level access to the regression model; `scater` for additional QC and normalisation diagnostics.

For Reviewers

What to look for in a paper using this method.

- Common misapplications.
- Diagnostics that should be reported but often aren't.
- Red flags in tables and figures.
- What to verify.
- What an adequate Methods paragraph must contain.

See also — labs in this chapter

- RNA-seq with DESeq2 / edgeR
- Enrichment analysis (GSEA / ORA)
- scRNA-seq with Seurat